

**HYPERLOCAL URBAN HEAT VULNERABILITY MAPPING AND
ALERT SYSTEM
PROJECT REPORT**

Submitted by

DEEPIKA.B	511522205010
INDUMATHI.Y	511522205020
LAKSHANA.K	511522205027
THAMIZHILAKKIYA.R	511522205054

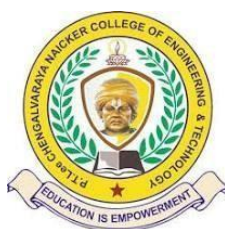
in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



**P.T. Lee CHENGALVARAYA NAICKER
COLLEGE OF ENGINEERING AND TECHNOLOGY
Ooverly , Kanchipuram-631 502.**



ANNA UNIVERSITY::CHENNAI- 600 025

MAY 2026

ANNA UNIVERSITY:CHENNAI- 600 025

BONAFIDE CERTIFICATE

Certified that this project report **“HYPERLOCAL URBAN HEAT VULNERABILITY MAPPING AND ALERT SYSTEM”** is the bonafide work of **“DEEPIKA.B (511522205010), INDUMATHI (511522205020), LAKSHANA.K(511522205027), THAMIZHILAKKIYA.R(511522205054”** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported here in does not from part of another report or dissertation on the basis of which a degree of award was conferred on an earlier occasion on this on another candidate.

SIGNATURE

**Mr. E. KAMALANATHAN, M.E.,
HEAD OF THE DEPARTMENT
ASSISTANT PROFESSOR**

Department of Information Technology
P.T. Lee CNCET
Kanchipuram-631 502

SIGNATURE

**Mr. E. KAMALANATHAN, M.E.,
SUPERVISOR
ASSISTANT PROFESSOR**

Department of Information Technology
P.T. Lee CNCET
Kanchipuram-631 502

Submitted for the ANNA UNIVERSITY examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We record our sincere thanks to Almighty GOD and our beloved parents who provided us this chance during our tenure in college. We are grateful to our college & our beloved principal **Dr. P. PALANISAMY, M.E., Ph.D.**, for been constantly supporting all our endeavours.

We are also thankful to **Mr. E. KAMALANATHAN, M.E.**, Head of the Department and Project Coordinator of Computer Science and Engineering for providing the necessary facilities during the execution of our project work. We also thank for his valuable suggestions, advice, guidance and constructive ideas in each and every step, which was indeed a great need towards the successful completion of the project.

I express my whole hearted thanks to the **Director, Dr. M. ARULARASU, M.E., Ph.D.** for his excellent suggestions and mentoring us in our academic achievements.

This project would not have been a success without my internal guide. So, I would extend my deep sense of gratitude to my Internal guide / Supervisor **Mr. E. KAMALANATHAN, M.E.**, for the effort he/she took in guiding us in all the stages of completion of my project work.

We would also like to thank all other faculty members of the Department of Computer Science and Engineering for their immense help throughout this project.

DEEPIKA.B

INDUMATHI.Y

LAKSHANA.K

THAMIZHILAKKIYA. R

ABSTRACT

This project presents the design and development of a Hyperlocal Urban Heat Vulnerability Mapping and Alert System using machine learning and geospatial technologies. The system analyzes temperature variations at a fine spatial level using satellite-derived data such as NDVI and NDBI.

A Random Forest Regression model is used to predict temperature based on geographical and environmental features. The system provides interactive heatmap visualization using Folium and allows users to explore temperature distribution across different regions.

An alert mechanism is integrated to notify users when temperature exceeds predefined thresholds, helping identify heat-vulnerable zones. The application is developed using Streamlit, ensuring a user-friendly interface and real-time interaction.

The system aims to support urban planning, environmental monitoring, and decision-making by providing accurate and localized heat analysis.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	Acknowledgement	ii
	Abstract	iii
	List of Figures	viii
	List of Abbreviation	ix
1.	INTRODUCTION	1
	1.1 Objective	2
	1.2 Scope	2
2.	LITERATURE REVIEW	3
3.	SYSTEM ANALYSIS	5
	3.1 Feasibility Study	5
	3.1.1 Technical Feasibility	5
	3.1.2 Economic Feasibility	6
	3.1.3 Operational Feasibility	6
	3.1.4 User Interface Requirements	6
	3.1.5 Intelligent Agents	7
	3.2 Existing System	7
	3.3 Proposed System	8
	3.4 Dataset Description	8

	3.4.1	Data Source	8
	3.4.2	Dataset Format	8
	3.4.3	Dataset Attributes	9
	3.4.4	Data Collection Process	9
	3.4.5	Data Preprocessing	9
	3.5	System Architecture Design	10
4.		SOFTWARE DESCRIPTION	11
	4.1	Streamlit	11
	4.2	Folium	11
	4.3	Google Earth Engine	12
	4.4	Scikit-learn	12
5.		ALGORITHM IMPLEMENTATION	13
	5.1	Random Forest Regression	13
	5.2	Model Evaluation	14
	5.3	Alert Algorithm	16
6.		SYSTEM DESIGN	19
	6.1	Data Flow Diagram	19
	6.2	Use case Diagram	20
	6.3	Class Diagram	21
	6.4	Sequence Diagram	22

6.5	Communication Diagram	23
6.6	Deployment Diagram	24
7.	INPUT DESIGN AND OUTPUT DESIGN	25
7.1	Input Design	25
7.2	Output Design	26
8.	MODULES DESCRIPTION	29
8.1	Data Collection Module	29
8.1.1	Data Extraction using GEE	29
8.1.2	Functions	30
8.1.3	Code	30
8.2	Data Processing Module	33
8.3	Machine Learning Module	34
8.4	Visualization Module	34
8.5	Alert Module	35
9.	SYSTEM TESTING	44
9.1	Unit Testing	44
9.2	Integration Testing	45
10.	SYSTEM IMPLEMENTATION	46
11.	CONCLUSION	47
	& FUTURE ENHANCEMENT	

	11.1	Conclusion	47
	11.2	Future Enhancement	47
12.		BIBLIOGRAPHY	49
13.		REFERENCES	50

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.5	System Architecture Design	10
6.1	Data Flow Diagram	19
6.2	Use Case Diagram	20
6.3	Class Diagram	21
6.4	Sequence Diagram	22
6.5	Communication Diagram	23
6.6	Deployment Diagram	24
7.1	Heatmap Visualization	27
7.2	Alerts System	28
8.1	Getting Input from User	42
8.2	Heatmap for Study Area	42
8.3	Vulnerability score & Alert Message	43
9.1	Unit Testing	45
9.2	Integration Testing	45

LIST OF ABBREVIATION

UHI	Urban Heat Island
GEE	Google Earth Engine
NDVI	Normalized Difference Vegetation Index
NDBI	Normalized Difference Built-up Index
LST	Land Surface Temperature
ML	Machine Learning
RF	Random Forest
MAE	Mean Absolute Error
GIS	Geographic Information System
CSV	Comma Separated Values
UI	User Interface

CHAPTER 1

INTRODUCTION

Urbanization has led to significant environmental challenges, one of the most critical being the Urban Heat Island (UHI) effect. This phenomenon refers to the increase in temperature observed in urban areas compared to surrounding rural regions. The primary reasons for this effect include dense construction, excessive use of concrete and asphalt, reduced vegetation cover, and continuous human activities such as transportation and industrial operations. These factors contribute to the absorption and retention of heat, resulting in elevated temperatures, especially during daytime and even persisting at night.

The rapid growth of cities has replaced natural landscapes such as forests, water bodies, and open land with built-up structures. Materials like concrete and asphalt have high thermal capacity, meaning they absorb heat during the day and release it slowly over time. In addition, the reduction of green spaces limits natural cooling processes such as evapotranspiration. As a result, urban areas experience higher temperatures, leading to discomfort and serious environmental concerns.

In recent years, the impact of climate change has further intensified this issue. Heatwaves are becoming more frequent and severe, posing serious risks to human health, including heatstroke, dehydration, and respiratory problems. Moreover, increased temperatures lead to higher energy consumption due to the excessive use of air conditioning systems, which in turn contributes to increased greenhouse gas emissions. This creates a cycle that further worsens the urban heat problem.

Traditional weather monitoring systems are not sufficient to address this issue effectively. These systems rely on a limited number of meteorological stations that provide data at a coarse spatial resolution.

However, to overcome these limitations, this project introduces a hyperlocal urban heat vulnerability mapping system. The concept of hyperlocal analysis focuses on studying environmental conditions at a very fine spatial resolution, enabling detailed insights into temperature variations across small geographic areas. This approach helps in identifying specific zones that are more vulnerable to extreme heat conditions.

The system is developed as a web-based application using Streamlit, ensuring that it is user-friendly and accessible even to non-technical users. By combining

advanced technologies with an intuitive interface, the project provides a practical and efficient solution for urban heat analysis.

Overall, this project aims to bridge the gap between large-scale environmental monitoring and localized urban planning.

1.1 Objective

The main objective of this project is to develop a system that can:

- Analyze temperature variations at a hyperlocal level
- Predict temperature using machine learning
- Visualize heat distribution using maps
- Identify vulnerable zones
- Provide alerts for extreme heat conditions

1.2 Scope

The scope of the project includes:

- Mapping temperature distribution for specific regions (e.g., Kanchipuram)
- Using satellite data for analysis
- Developing a web-based application

Limitations:

- Depends on satellite data availability
- Accuracy depends on model performance

CHAPTER 2

LITERATURE REVIEW

- [1] F. Larroya et al., “Hyperlocal Urban Heat Mapping using Satellite Data,” *Remote Sensing of Environment*, vol. 310, 2025.**

This study focuses on hyperlocal urban heat mapping by integrating satellite imagery with ground-level temperature measurements. The authors demonstrate that significant temperature variations can occur even within small urban regions, emphasizing the importance of fine-scale spatial analysis. By combining high-resolution remote sensing data with in-situ observations, the study improves the accuracy of heat distribution mapping. The findings highlight the need for targeted, location-specific mitigation strategies to effectively address urban heat hotspots and enhance climate resilience in cities.

- [2] Institute of Electrical and Electronics Engineers, “Machine Learning in Heat Vulnerability Mapping,” *IEEE Access*, vol. 11, pp. 56789–56805, 2023.**

This study explores the application of machine learning algorithms in predicting heat vulnerability using satellite-derived and socio-economic datasets. Techniques such as Random Forest and Neural Networks are employed to identify complex patterns and enhance prediction accuracy. The research demonstrates that AI-driven approaches are capable of handling large-scale datasets and enabling near real-time analysis. It further highlights the growing role of intelligent systems in smart city initiatives, particularly for heat risk assessment and urban planning.

- [3] World Health Organization, “Heat Risk Early Warning Systems,” *WHO Climate and Health Technical Report Series*, 2023.**

This research examines the development of heat alert systems aimed at minimizing the impact of extreme heat events on public health. The

system integrates weather forecasts, satellite observations, and heat vulnerability indices to generate timely warnings. These alerts enable governments and communities to implement preventive measures such as opening cooling centers and issuing public advisories. The study underscores the critical role of early warning systems in reducing heat-related mortality and improving overall community preparedness.

[4] P. Siddiqui et al., “Urban Heat Vulnerability Mapping using GIS and Remote Sensing,” *Sustainable Cities and Society*, vol. 85, 2022.

This research combines Geographic Information Systems (GIS) with remote sensing data to develop detailed urban heat vulnerability maps. The study integrates Land Surface Temperature (LST), vegetation indices, and socio-economic indicators to assess vulnerability at a fine spatial scale. Results indicate that areas with high population density and limited green cover are more susceptible to heat stress. The authors emphasize the importance of integrating environmental and social parameters for accurate vulnerability assessment and highlight the role of GIS tools in spatial visualization and informed decision-making.

[5] S. C. Sheridan and L. S. Kalkstein, “Heat Vulnerability Index: Advancements and Applications,” *International Journal of Environmental Research and Public Health*, vol. 19, no. 14, 2022.

This study presents modern advancements in the Heat Vulnerability Index (HVI) as a comprehensive tool for identifying populations at risk from extreme heat events. The index incorporates demographic, socioeconomic, and environmental variables such as age distribution, income levels, population density, access to cooling infrastructure, and healthcare availability. Recent developments enhance the model by integrating high-resolution spatial data, satellite-derived temperature information, and GIS-based analysis for improved accuracy at the hyperlocal level. The research highlights that heat vulnerability varies significantly across urban areas due to differences in infrastructure, land use patterns, and socioeconomic conditions.

CHAPTER 3

SYSTEM ANALYSIS

System analysis is an important phase in the development of any project, as it evaluates the feasibility, identifies existing system limitations, and defines the proposed solution. This chapter provides a detailed analysis of the system along with feasibility studies and architecture design.

3.1 Feasibility Study

Feasibility analysis determines whether the proposed system is practical and can be successfully implemented. It includes technical, economic, and operational feasibility.

3.1.1 Technical Feasibility

The proposed system is technically feasible as it utilizes widely available and well-supported technologies. The implementation is based on Python, which is a powerful programming language with extensive libraries for data analysis and machine learning.

The system uses:

- Python for core development
- Streamlit for building the user interface
- Google Earth Engine for satellite data processing
- Folium for map visualization

These technologies are reliable, scalable, and capable of handling the requirements of the project. Additionally, they are well-documented and supported by strong developer communities, making development and troubleshooting easier.

3.1.2 Economic Feasibility

The project is economically feasible as it primarily relies on open-source tools and platforms.

Key points:

- No licensing cost for software
- No requirement for expensive hardware
- Minimal operational cost

Since the system uses free tools such as Python, Streamlit, and Google Earth Engine, the overall development cost is significantly reduced. This makes the project cost-effective and suitable for academic and practical implementation.

3.1.3 Operational Feasibility

The system is designed to be user-friendly and accessible to both technical and non-technical users.

Key features include:

- Simple and intuitive interface
- Easy navigation and interaction
- Minimal training required

Users can easily input data, view heatmaps, and receive alerts without requiring specialized knowledge. This ensures smooth operation and usability of the system.

3.1.4 User Interface Requirements

The user interface plays a crucial role in enhancing user experience and system usability. The following requirements are considered:

- Simple and Interactive Dashboard:
Provides a clean and organized layout for easy interaction
- Map-Based Visualization:

Displays temperature distribution using interactive maps

- **User Input Options:**
Allows users to enter parameters such as location and threshold values
- **Alert Display:**
Shows warning messages when temperature exceeds limits

The interface is designed using Streamlit to ensure responsiveness and real-time updates.

3.1.5 Intelligent Agents

The system incorporates machine learning as an intelligent agent to automate analysis and decision-making.

Functions of intelligent agent:

- Predicts temperature values based on input data
- Identifies heat-vulnerable zones
- Triggers alerts for extreme conditions

The use of machine learning enhances the system's capability to provide accurate and efficient results without manual intervention.

3.2 Existing System

The existing systems for temperature monitoring include:

- Traditional weather monitoring stations
- Satellite-based analysis tools without user interaction

Limitations of Existing System

- Low spatial resolution
- Lack of predictive capabilities

3.3 Proposed System

The proposed system addresses the limitations of existing systems by introducing advanced technologies and features.

Key features include:

- Use of machine learning for accurate prediction
- Hyperlocal mapping for detailed analysis
- Interactive heatmap visualization
- Integrated alert system

3.4 Dataset Description

The dataset used in this project is a structured collection of geospatial, environmental, and temporal data, designed to analyze and predict urban heat vulnerability at a hyperlocal level. The dataset is stored in a CSV file named **temperature_data.csv** and is used for training and testing the machine learning model.

3.4.1 Data Source

The dataset is generated using satellite imagery obtained from Google Earth Engine (GEE). It primarily utilizes Sentinel-2 satellite data to extract environmental features such as vegetation and built-up indices, which are essential for temperature analysis.

3.4.2 Dataset Format

The dataset is stored in CSV (Comma-Separated Values) format, making it easy to process using Python libraries such as Pandas and Scikit-learn. Each record in the dataset represents a specific geographic location along with its associated environmental parameters.

3.4.3 Dataset Attributes

The dataset consists of the following attributes:

- **Latitude_x** – Latitude coordinate of the location
- **Longitude_x** – Longitude coordinate of the location
- **NDVI (Normalized Difference Vegetation Index)** – Represents vegetation density
- **NDBI (Normalized Difference Built-up Index)** – Indicates built-up area intensity
- **Year** – Year of data collection
- **Time** – Encoded time of the day (1: Morning, 2: Afternoon, 3: Evening)
- **Temp_C** – Temperature in degrees Celsius (Target variable)

3.4.4 Data Collection Process

The dataset is created through the following steps:

- Selection of region of interest (e.g., Kanchipuram area)
- Extraction of satellite images using Google Earth Engine
- Filtering images based on cloud coverage and date
- Calculation of NDVI and NDBI indices
- Extraction of latitude and longitude values
- Conversion of geospatial data into tabular CSV format

3.4.5 Data Preprocessing

Before training the model, the dataset undergoes preprocessing to improve quality and accuracy:

- Handling missing or null values
- Feature selection
- Encoding time values into numerical format

3.4.6 Role of Dataset in the System

The dataset plays a vital role in the system by:

- Training the Random Forest Regression model
- Predicting temperature values
- Supporting heatmap visualization
- Identifying vulnerable zones
- Triggering alerts based on temperature thresholds

3.4.7 Limitations of Dataset

- Depends on availability of satellite data
- May contain noise due to atmospheric conditions
- Limited temporal resolution

Model accuracy depends on data quality

3.5 System Architecture Design

The system architecture defines the overall workflow and interaction between different compounds

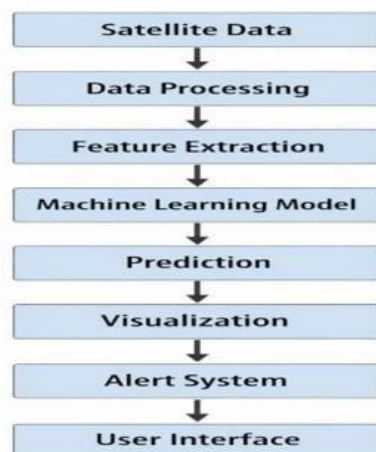


Fig 3.5 System Architecture Design

CHAPTER 4

SOFTWARE DESCRIPTION

4.1 Streamlit

Streamlit is an open-source Python framework used for developing interactive and user-friendly web applications, especially for data science and machine learning projects. In this project, Streamlit serves as the frontend interface that allows users to interact with the system easily without requiring advanced technical knowledge.

Streamlit enables rapid development by converting Python scripts directly into web applications. It provides built-in components such as sliders, buttons, text inputs, and data visualization tools, which are used to display results dynamically. In this system, Streamlit is used to present heatmaps, display predicted temperature values, and show alert messages when temperature thresholds are exceeded.

One of the major advantages of Streamlit is its simplicity and real-time updating capability. Any changes in the code are instantly reflected in the application, making it highly efficient for development and testing. Additionally, it integrates seamlessly with data processing libraries and visualization tools, making it ideal for this project.

4.2 Folium

Folium is a powerful Python library used for creating interactive maps. It is built on top of the Leaflet.js library and is widely used for geospatial data visualization. In this project, Folium is used to generate interactive maps and visualize temperature distribution using heatmaps.

Folium allows the integration of geographic data with mapping tiles from OpenStreetMap, enabling users to explore spatial patterns effectively. This makes it easy to identify high-temperature zones and vulnerable areas.

Additional features of Folium include marker placement, layer control, and custom map styling.

4.3 Google Earth Engine

Google Earth Engine (GEE) is a cloud-based geospatial processing platform that provides access to a vast repository of satellite imagery and environmental datasets. It is a key component of this project, as it supplies the raw data required for temperature analysis.

GEE allows users to process large-scale geospatial data efficiently using its powerful computational infrastructure. Bands like B4, B5, B8, and B10 are used to study vegetation, reflectance, and thermal properties of the Earth's surface.

The platform also supports advanced data processing techniques such as filtering, masking, and image reduction, which are essential for preparing the dataset. By using GEE, the project benefits from high-quality, up-to-date satellite data without the need for local storage or heavy computation.

4.4 Scikit-learn

Scikit-learn is a widely used machine learning library in Python that provides simple and efficient tools for data analysis and modeling. In this project, Scikitlearn is used to implement the Random Forest Regression algorithm for temperature prediction.

The library offers a variety of functionalities including data preprocessing, model training, evaluation, and validation. It simplifies the process of splitting data into training and testing sets, fitting the model, and evaluating its performance using metrics such as Mean Absolute Error (MAE).

Random Forest, an ensemble learning method, is chosen due to its high accuracy and ability to handle complex datasets. Scikit-learn makes it easy to implement this algorithm and optimize its performance. The library is also welldocumented and integrates seamlessly with other Python tools, making it highly suitable for this project.

CHAPTER 5

ALGORITHM IMPLEMENTATION

5.1 Random Forest Regression

Random Forest Regression is an advanced machine learning algorithm that belongs to the category of ensemble learning methods. Ensemble learning involves combining multiple models to improve overall performance and accuracy. In the case of Random Forest, it combines several decision trees to produce a more reliable and robust prediction.

A decision tree is a supervised learning algorithm that splits the dataset into smaller subsets based on feature values. However, a single decision tree may suffer from overfitting, where it performs well on training data but poorly on unseen data. Random Forest overcomes this limitation by constructing multiple decision trees and aggregating their outputs.

In this project, Random Forest Regression is used to predict land surface temperature based on features extracted from satellite data, such as spectral and thermal bands.

Working Principle:

The working of Random Forest involves the following key concepts:

- **Bootstrap Sampling:** Random subsets of the dataset are created using sampling with replacement.
- **Random Feature Selection:** At each split in a tree, a random subset of features is selected instead of considering all features.
- **Ensemble Prediction:** The final prediction is obtained by averaging the outputs of all decision trees.

Algorithm Steps

1. Data Collection:

Gather input features such as spectral bands (B4, B5, B8, B10) from satellite data.

2. Data Splitting:

Divide the dataset into training and testing sets.

3. Random Sampling:

Create multiple subsets of the training data using bootstrap sampling.

4. Tree Construction:

For each subset, build a decision tree by selecting random features at each node and splitting the data accordingly.

5. Prediction:

Each decision tree produces a prediction for the given input.

6. Aggregation:

Combine all predictions by calculating the average value, which becomes the final predicted temperature.

Advantages of Random Forest

- **High Accuracy:** Combines multiple models to improve prediction accuracy.
- **Reduces Overfitting:** Averaging results reduces variance.
- **Robust to Noise:** Performs well even with noisy or incomplete data.
- **Handles Large Datasets:** Efficient with high-dimensional data.
- **Feature Importance:** Helps identify which features contribute most to predictions.

5.2 Model Evaluation

After training the machine learning model, it is essential to evaluate its performance to ensure that it produces accurate and reliable predictions. In this project, the performance of the Random Forest model is evaluated using Mean Absolute Error (MAE).

Mean Absolute Error (MAE)

Mean Absolute Error is a widely used evaluation metric for regression models. It measures the average absolute difference between the predicted values and the actual values.

Formula

MAE is calculated as:

$$[\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|]$$

Where:

- (y_i) = Actual value
- $(\hat{y}_i)$ = Predicted value
- (n) = Number of observations

Explanation

- MAE calculates the magnitude of errors without considering their direction.
- It provides a simple and interpretable measure of model performance.
- A lower MAE value indicates better prediction accuracy.

Importance in This Project

- Helps evaluate how accurately the model predicts temperature
- Ensures reliability of heat vulnerability mapping
- Assists in improving model performance

5.3 Alert Algorithm

The Alert Algorithm is an essential component of the system, designed to notify users when temperature levels exceed a predefined threshold. This helps in identifying high-risk areas and taking preventive measures.

Purpose of Alert System

- To warn users about extreme temperature conditions
- To highlight vulnerable zones
- To support decision-making for safety and planning

Working Principle

The alert system continuously monitors predicted temperature values generated by the machine learning model. When the predicted value crosses a specified threshold, an alert is triggered and displayed in the user interface.

Algorithm Steps

1. Define Threshold Value:
Set a temperature limit (e.g., 35°C or higher) based on risk levels.
2. Input Predicted Temperature:
Receive predicted temperature from the Random Forest model.
3. Comparison:
Compare the predicted value with the threshold.
4. Condition Check:
 - If predicted temperature $\leq$ threshold $\rightarrow$ No alert
 - If predicted temperature $>$ threshold $\rightarrow$ Trigger alert
5. Alert Generation:
Display warning message in the Streamlit interface.

6. Visualization:

Highlight affected regions in heatmap (e.g., red color).

Types of Alerts

- Normal: Safe temperature levels
- Warning: Moderate risk
- Critical Alert: High-risk temperature zones

Advantages of Alert Algorithm

The alert system is a crucial component of the Hyperlocal Urban Heat Vulnerability Mapping and Alert System. It enhances the practical usability of the application by providing timely warnings and improving user awareness. The major advantages of the alert system are as follows:

1.Real-Time Notification

The system provides immediate alerts when the predicted temperature exceeds the predefined threshold. This helps users take quick action to prevent heatrelated risks.

2. Early Risk Identification

By continuously monitoring temperature values, the alert system identifies highrisk zones at an early stage. This allows authorities and individuals to respond proactively rather than reactively.

3. Improved Decision-Making

The alerts assist urban planners, government agencies, and the public in making informed decisions regarding heat management, resource allocation, and safety measures.

4. Enhanced Public Safety

The system helps reduce health risks such as heatstroke, dehydration, and fatigue by warning users about extreme temperature conditions in advance.

5. User Awareness

Alerts increase awareness about environmental conditions and encourage users to adopt preventive measures such as staying hydrated, avoiding outdoor activities during peak heat, and using protective measures.

6. Integration with Visualization

The alert system works in coordination with heatmaps, highlighting critical regions in red or high-intensity colors. This visual representation makes it easier to understand the severity of the situation.

7. Customizable Thresholds

Users can set temperature thresholds based on their requirements. This flexibility allows the system to be adapted for different regions and use case.

CHAPTER 6

SYSTEM DESIGN

6.1 Data Flow Diagrams

Data flow diagrams are useful in modelling high-level systems and processes. These diagrams show the way in which data flows between elements in a system and between the system and external entities.

DFD

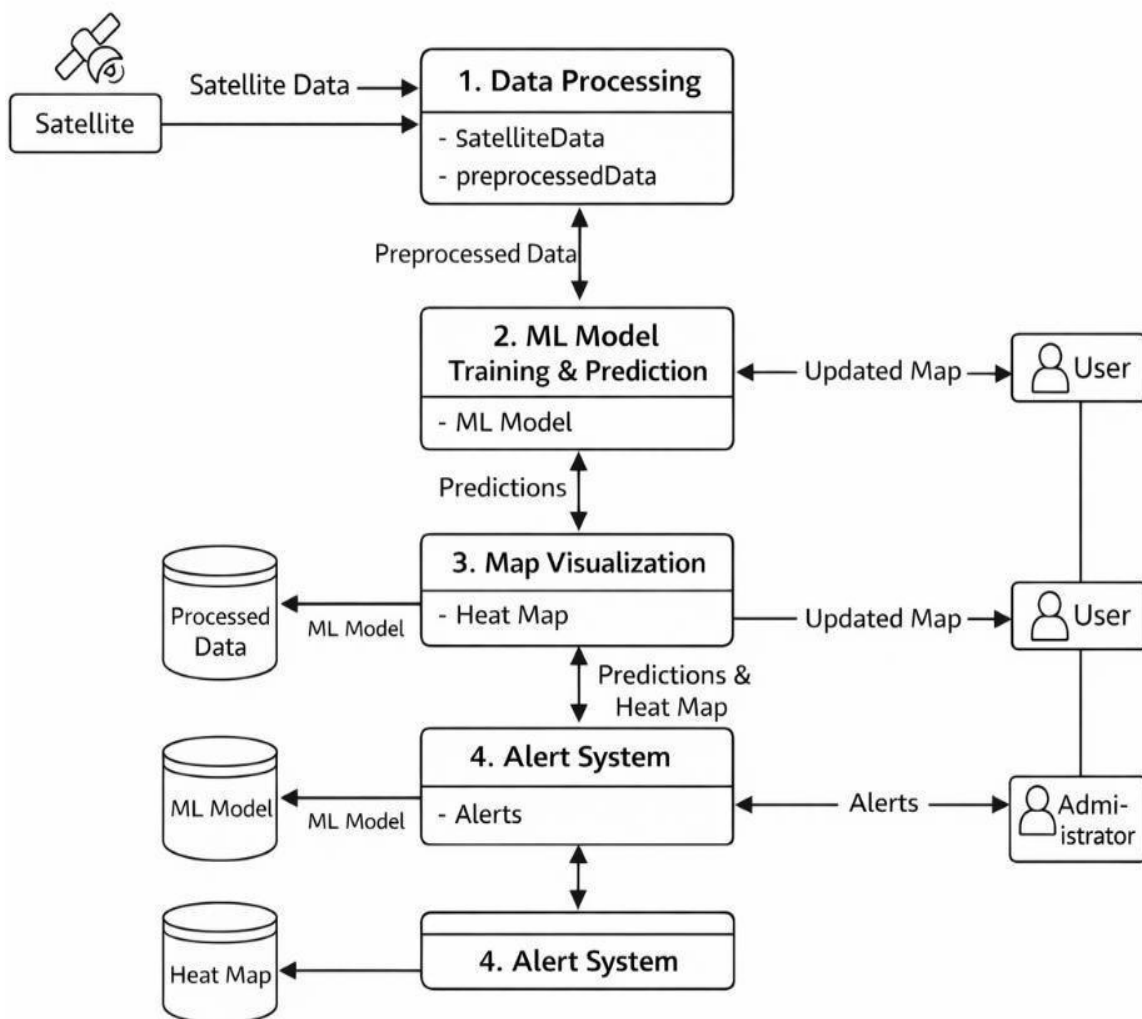


Fig 6.1 Data Flow Diagram

6.2 Use-Case Diagram

A use-case diagram presents a collection of use cases and actors and is typically used to specify or characterize the functionality and behaviour of a whole application system interacting with one or more external actors.

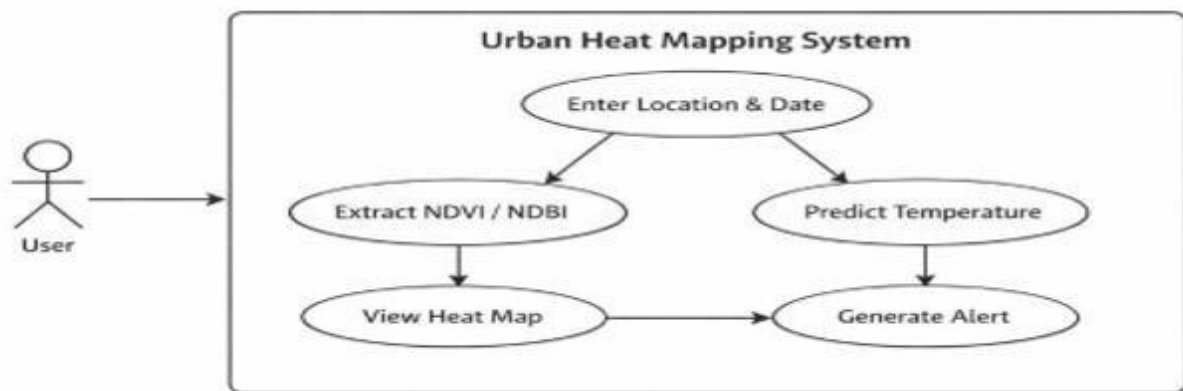


Fig 6.2 Use-Case Diagram

6.3 Class Diagram

A class describes the structure of the systems in terms of classes and objects. Classes are the abstraction that specify the behaviours of set of objects are entities that encapsulation state and behaviour.

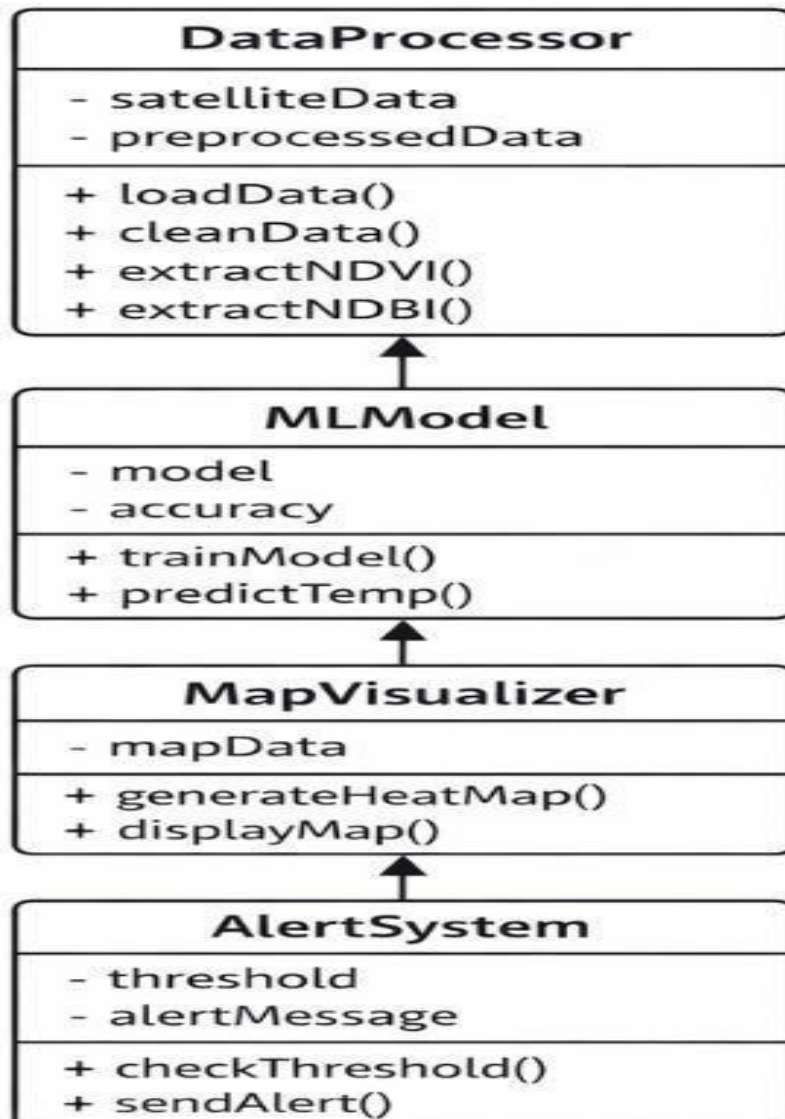


Fig 6.3 Class Diagram

6.4 Sequence Diagram

A sequence diagram traces the execution of an interaction in time. Message sequence numbers are automatically generated based on the message's vertical order. To change a message's position in the sequence, use the left mouse button to drag it to the desired position; message sequence numbers will automatically be recomputed to reflect the new order.

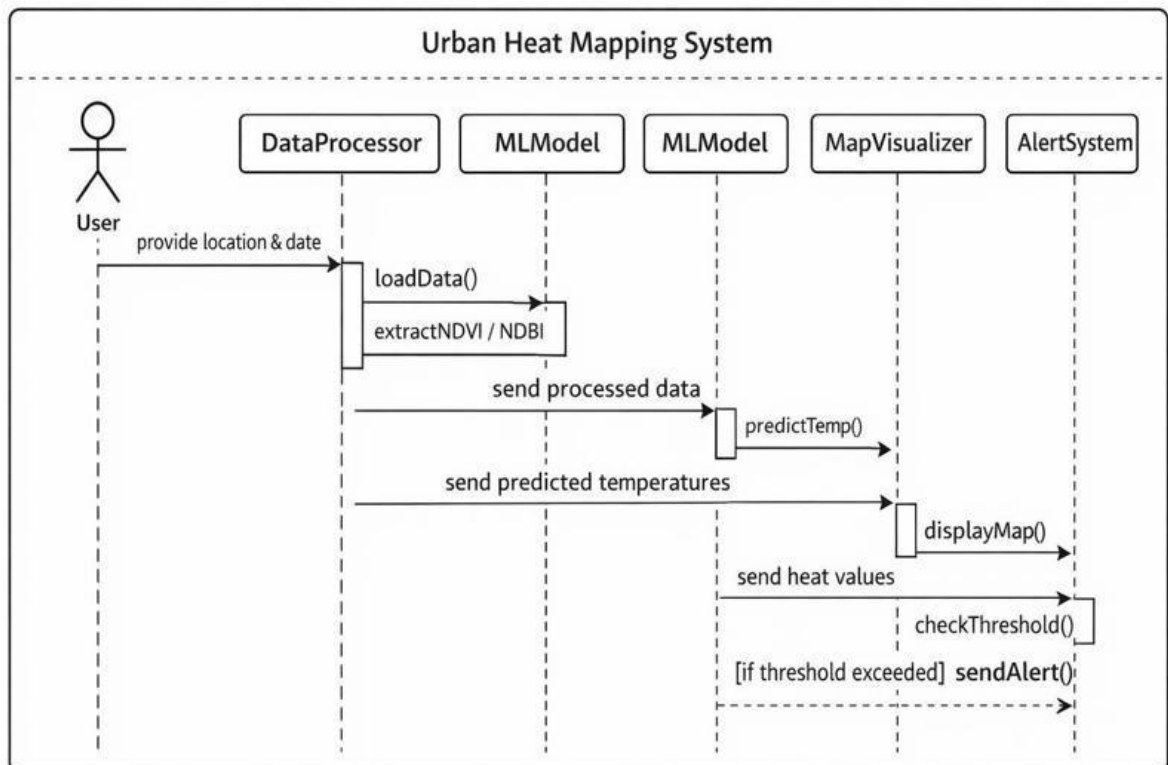


Fig 6.4 Sequence Diagram

6.5 Communication Diagram

Communication diagram is also used to model the dynamic behaviour of the system. When compared to sequence diagram, the communication diagram is more focused on showing the collaboration of objects rather than the time sequence.

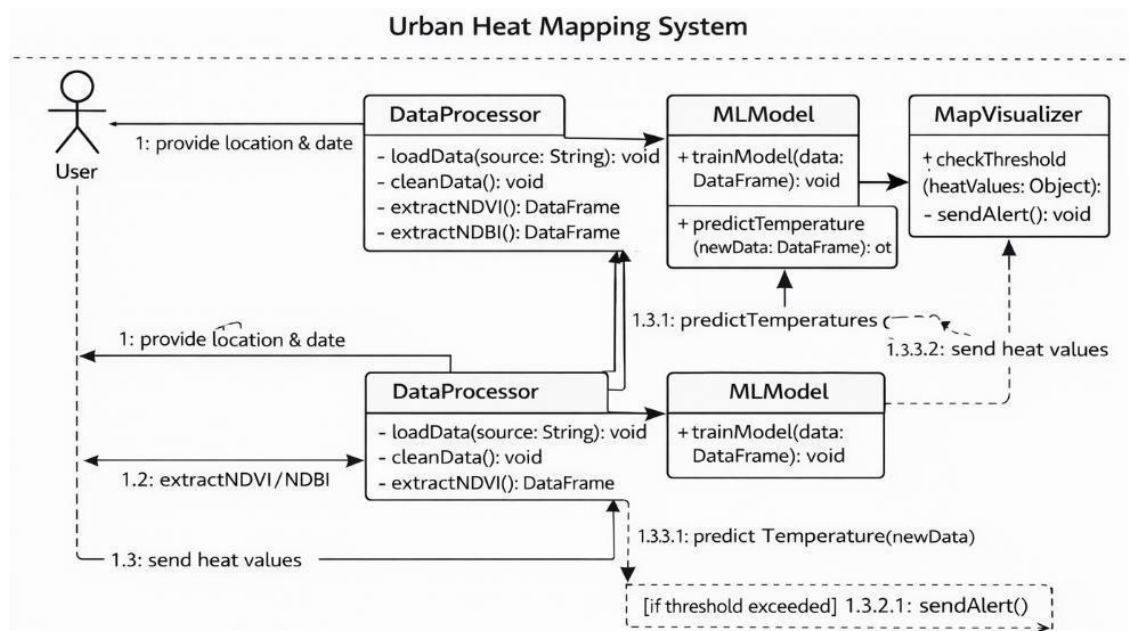


Fig 6.5 Communication Diagram

6.6 Deployment Diagram

Deployment diagram is defined as assignment of concrete software artifacts to computational nodes. In other words, deployment of software elements to the physical architecture and the communication between physical elements.

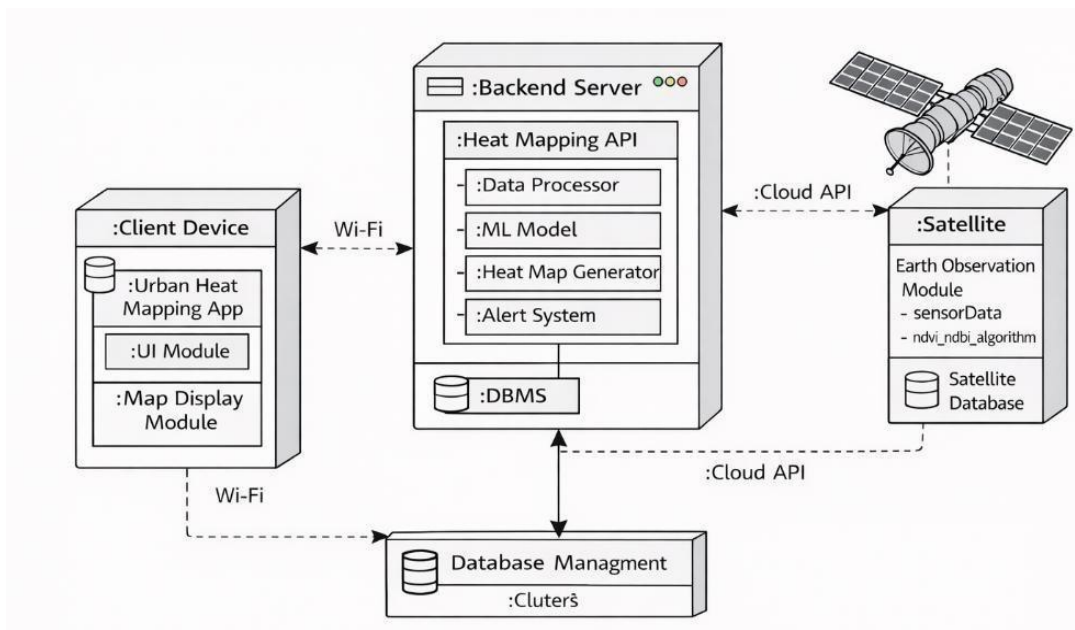


Fig 6.6 Deployment Diagram

CHAPTER 7

INPUT AND OUTPUT DESIGN

7.1 Input Design

Input design is a crucial phase in system development, as it defines how data is collected, processed, and fed into the system. A well-designed input system ensures accuracy, efficiency, and reliability of the overall application. In this project, multiple types of inputs are used to analyze and predict urban heat vulnerability.

1. Satellite Data

Satellite data forms the primary input for this system. It is obtained using Google Earth Engine and includes various spectral and thermal bands that are essential for analyzing land surface characteristics.

Key satellite inputs include:

- Thermal Band (B10): Used to estimate land surface temperature
- Red Band (B4): Helps in analyzing vegetation and surface properties
- Near Infrared Bands (B5, B8): Used to assess vegetation health and reflectance

These bands provide valuable information about temperature, vegetation cover, and surface conditions. The data is preprocessed to remove noise and convert it into a structured format suitable for machine learning.

2. Coordinates

Geographical coordinates (latitude and longitude) are used to specify the exact location for analysis. These coordinates help in:

- Mapping specific regions
- Generating location-based predictions
- Visualizing data accurately on maps

Users can input coordinates manually or select a region directly from the map interface. This allows the system to perform hyperlocal analysis.

3. User Parameters

User-defined parameters allow customization of the system based on specific requirements. These parameters include:

- Temperature threshold for alerts
- Region selection
- Visualization preferences

By adjusting these inputs, users can tailor the analysis according to their needs. This improves flexibility and usability of the system.

Input Design Considerations

- Inputs are validated to ensure correctness
- User interface is designed to be simple and intuitive
- Data is preprocessed before being used in the model
- Errors and missing values are handled effectively

7.2 Output Design

Output design focuses on how the processed data is presented to the user. The outputs should be clear, meaningful, and easy to interpret. In this project, outputs are presented through visual and textual formats to enhance user understanding.

1. Heatmap Visualization

The heatmap is the primary output of the system. It visually represents temperature distribution across different regions using color variations.

- Blue: Low temperature areas
- Green/Yellow: Moderate temperature
- Red: High temperature (vulnerable zones)

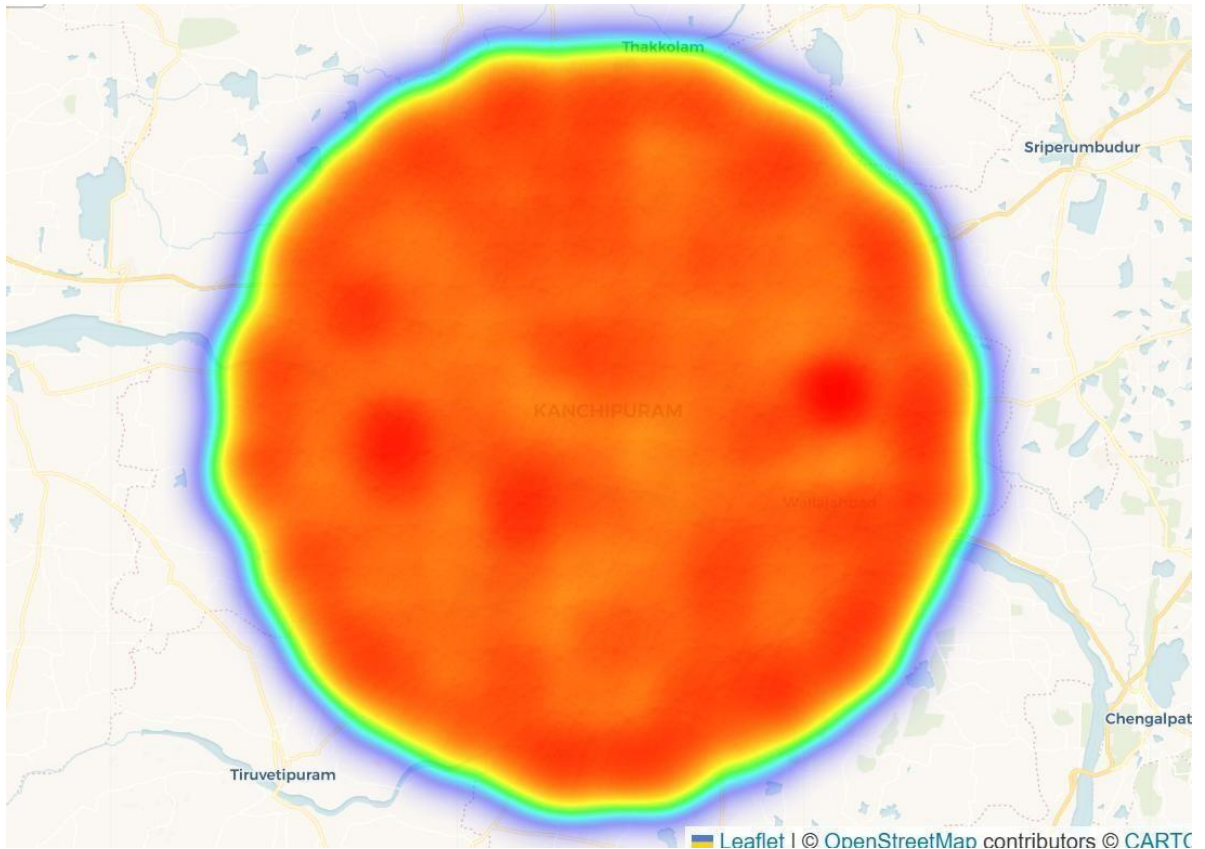


Fig: 7.1 Heatmap Visualization

The heatmap is generated using Folium and OpenStreetMap, allowing users to interact with the map through zooming and panning. This helps in identifying hotspots and understanding spatial patterns effectively.

2. Temperature Prediction

The system provides predicted temperature values for specific locations using the Random Forest model. These predictions are displayed numerically and are used to:

- Analyze temperature trends
- Identify high-risk areas
- Support decision-making

The accuracy of predictions is ensured through proper model training and evaluation.

3. Alert System

The alert system notifies users when temperature values exceed a predefined threshold. Alerts are displayed in the user interface and may include:

- Warning messages
- Highlighted regions on the map
- Notifications for critical conditions

This feature enhances the practical usability of the system by enabling users to take preventive measures.

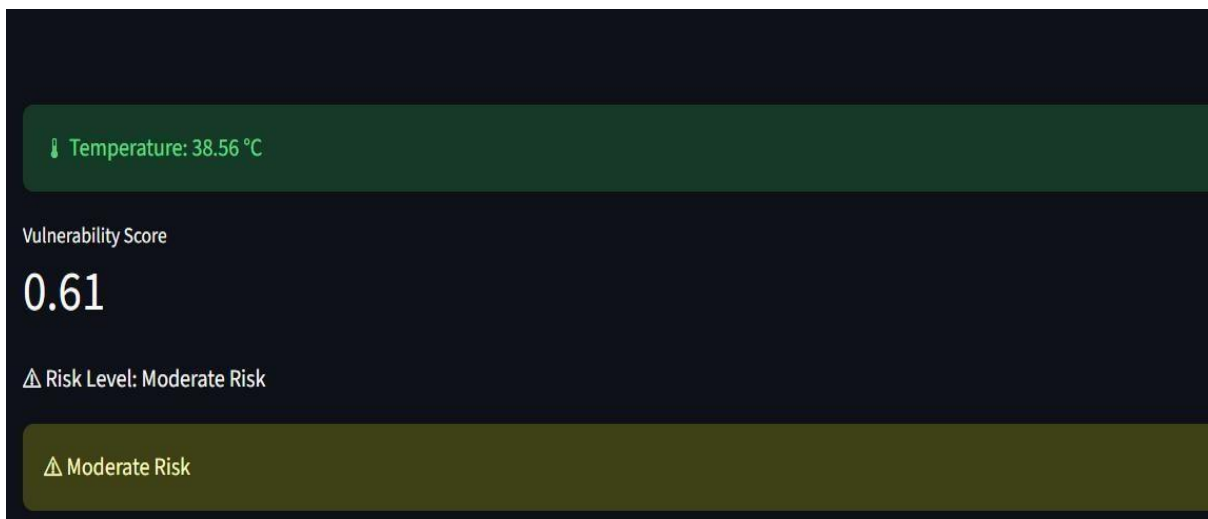


Fig: 7.2 Alerts System

CHAPTER 8

MODULES DESCRIPTION

The system is divided into several functional modules, each responsible for performing a specific task. Modular design improves the efficiency, scalability, and maintainability of the system. The major modules in this project are described below:

8.1 Data Collection Module

The Data Collection Module is responsible for gathering raw data required for analysis. In this project, satellite data is collected using Google Earth Engine, which provides access to large-scale geospatial datasets.

This module retrieves spectral and thermal band data such as B4 (Red), B5 (Near Infrared), B8 (NIR), and B10 (Thermal). These bands are essential for understanding land surface characteristics and temperature variations.

The module ensures that:

- Data is collected from reliable sources
- Required time range and region are selected
- Data is filtered based on quality parameters

The collected data serves as the foundation for further processing and analysis.

8.1.1 Data Extraction using Google Earth Engine

Google Earth Engine (GEE) is used in this project to collect satellite data for analyzing urban heat patterns. It provides access to large-scale datasets such as Landsat 8 and Sentinel-2.

In this module, satellite data is extracted for the Kanchipuram region by applying spatial and temporal filters. The data is filtered based on date (April– May) and cloud cover (<10%) to ensure accuracy.

Key features such as thermal bands, vegetation index (NDVI), and built-up index (NDBI) are calculated. The processed data is then converted into tabular format and exported as CSV files for machine learning.

8.1.2 Functions

- **getYearData(year):**
Extracts Landsat 8 data including thermal and spectral bands.
- **exportNDVI(year):**
Calculates NDVI (Normalized Difference Vegetation Index) using Sentinel-2 data.
- **exportNDBI(year):**
Calculates NDBI (Normalized Difference Built-up Index) for identifying urban areas.

8.1.3 CODE

```
// -----  
// STEP 1: DEFINE STUDY AREA  
// -----  
var kanchipuram = ee.Geometry.Point([79.70, 12.83]).buffer(20000);  
  
// -----  
// STEP 2: LANDSAT DATA EXTRACTION  
// -----function getYearData(year)  
{  
  
    var startDate = ee.Date.fromYMD(year, 4, 1); var  
    endDate      = ee.Date.fromYMD(year, 5, 31);  
  
    var dataset = ee.ImageCollection('LANDSAT/LC08/C02/T1_L2')  
        .filterBounds(kanchipuram)  
        .filterDate(startDate, endDate)  
        .filter(ee.Filter.lt('CLOUD_COVER', 10))  
}
```



```

    .mean()
    .clip(kanchipuram);          var          bands          =

dataset.select(['ST_B10', 'SR_B4', 'SR_B5']);

return bands.sample({
  region: kanchipuram,
  scale: 30, numPixels:
  5000, geometries:
  true
});
}

// Export Landsat data
Export.table.toDrive({ collection: getYearData(2023),
  description:
  'Landsat_Data_2023', fileNamePrefix:
  'Landsat_Data_2023', fileFormat:
  'CSV'
});

// -----
// STEP 3: NDBI CALCULATION
// ----- function
exportNDBI(year) {

  var startDate = ee.Date.fromYMD(year, 4, 1); var
endDate      = ee.Date.fromYMD(year, 5, 31);

  var s2 = ee.ImageCollection('COPERNICUS/S2_SR')
    .filterBounds(kanchipuram)
    .filterDate(startDate,                                     endDate)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10))
    .select(['B8', 'B11']) .median()

```

```

.clip(kanchipuram); var ndbi = s2.normalizedDifference(['B11',
'B8']) .rename('NDBI');

var samples =
  ndbi.sample({ region:
    kanchipuram, scale: 10,
    numPixels: 5000,
    geometries: true
  });

Export.table.toDrive({ collection: samples,
  description: 'Sentinel_NDBI_' + year,
  fileNamePrefix: 'Sentinel_NDBI_' + year, fileFormat:
  'CSV'
});
} exportNDBI(2023);

// -----
// STEP 4: NDVI CALCULATION
// ----- function
exportNDVI(year) {

  var startDate = ee.Date.fromYMD(year, 4, 1); var endDate
  = ee.Date.fromYMD(year, 5, 31);

  var s2 = ee.ImageCollection('COPERNICUS/S2_SR')
    .filterBounds(kanchipuram)
    .filterDate(startDate, endDate)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 10))
    .select(['B4', 'B8'])

    .median()
    .clip(kanchipuram); var ndvi = s2.normalizedDifference(['B8',
'B4']) .rename('NDVI');

```

```

var sample = ndvi.sample({
  region: kanchipuram,
  scale: 10, numPixels:
  5000, geometries: true
});

Export.table.toDrive({ collection: sample,
  description: 'Sentinel_NDVI_' + year,
  fileNamePrefix: 'Sentinel_NDVI_' + year,
  fileFormat: 'CSV'
});
} exportNDVI(2023);

```

8.2 Data Processing Module

The Data Processing Module is responsible for converting raw satellite data into a structured format suitable for machine learning.

Key functions include:

- Data cleaning (removal of missing or noisy values)
- Normalization and scaling
- Feature extraction from spectral bands
- Conversion of spatial data into tabular format

This module plays a crucial role in improving model accuracy by ensuring that the data is consistent and meaningful. Proper preprocessing helps in reducing errors and enhances the reliability of predictions.

8.3 Machine Learning Module

The Machine Learning Module is the core component of the system. It is responsible for training and implementing the predictive model.

In this project, a Random Forest Regression algorithm is used to predict temperature values based on input features. The module performs the following tasks:

- Splitting data into training and testing sets

- Training the model using historical data
- Predicting temperature for new inputs
- Evaluating model performance using MAE

The model learns patterns from the data and provides accurate predictions, which are essential for identifying heat vulnerability.

8.4 Visualization Module

The Visualization Module is responsible for presenting the results in a graphical and user-friendly manner. It uses tools like Folium and OpenStreetMap to create interactive maps.

Key features of this module include:

- Generation of heatmaps to represent temperature distribution
- Use of color gradients to indicate temperature intensity
- Interactive map controls (zoom, pan, layers)
- Display of markers and boundaries

This module helps users easily understand spatial patterns and identify hightemperature zones.

8.5 Alert Module

The Alert Module is designed to notify users about extreme temperature conditions. It enhances the practical usefulness of the system by providing realtime warnings.

Functions of this module:

- Define temperature threshold values
- Compare predicted temperature with threshold
- Trigger alerts when threshold is exceeded
- Display warning messages in the interface The alerts can be

categorized as:

- Normal (safe conditions)
- Warning (moderate risk)
- Critical (high-risk zones)

This module plays a vital role in helping users take preventive actions and reducing heat-related risks.

CODE:

```
import streamlit as st
import pandas as pd
import folium
import ee
import time

from streamlit_folium import st_folium
from folium.plugins import HeatMap

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score
# ----- #
Session State Init
# -----
if "prediction" not in st.session_state:
    st.session_state.prediction = None
if "time" not in st.session_state:
    st.session_state.time = None
# ----- #
Initialize GEE
# -----
try:
    ee.Initialize(project='urbanheatmapping')
except:
```

```

ee.Authenticate()
ee.Initialize(project='urbanheatmapping')

# ----- #
UI
# -----
st.set_page_config(layout="wide") st.title(" 📍 Hyperlocal Urban Heat
& Vulnerability Mapping System")

# ----- #
Load Data
# -----
@st.cache_data def
load_data():
    return pd.read_csv("temperature_data.csv")

data = load_data()

# ----- #
Train Model
# -----
@st.cache_resource def
train_model(data):
    features = data[[
        "Latitude_x", "Longitude_x", "NDVI", "NDBI", "Year", "Time"
    ]]

    target = data["Temp_C"]

    X_train, X_test, y_train, y_test = train_test_split(
        features, target, test_size=0.2, random_state=42
    )

    model = RandomForestRegressor(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

    pred = model.predict(X_test)

```

```

    return model, mean_absolute_error(y_test, pred), r2_score(y_test, pred)

model, mae, r2 = train_model(data)

col1, col2 = st.columns(2) col1.metric("MAE",
f"{mae:.2f} °C") col2.metric("R2 Score",
f"{r2:.2f}")

# ----- #
Functions
# -----
def encode_time(slot):
    return 1 if "Morning" in slot else 2 if "Afternoon" in slot else 3

def classify_heat(temp):
    return "High" if temp > 45 else "Medium" if temp > 40 else "Low"

def calculate_vulnerability(temp, ndvi, ndbi):
    return round((0.4*(temp/50) + 0.3*(1-ndvi) + 0.3*ndbi), 2)

def classify_risk(v):
    return "High Risk" if v > 0.7 else "Moderate Risk" if v > 0.4 else "Low Risk"

def get_indices(lat, lon, date):

    point = ee.Geometry.Point([lon, lat])
    start = ee.Date(str(date))    end =
    start.advance(15, 'day')

    collection = (
        ee.ImageCollection("COPERNICUS/S2_SR")
        .filterBounds(point)
        .filterDate(start, end)
        .filter(ee.Filter.lt("CLOUDY_PIXEL_PERCENTAGE", 20))
    )

```

```

if collection.size().getInfo() == 0:
    return 0, 0

image = collection.median()

nir = image.select("B8")
red = image.select("B4")
swir = image.select("B11")

ndvi = nir.subtract(red).divide(nir.add(red))
ndbi = swir.subtract(nir).divide(swir.add(nir))

ndvi_val = ndvi.reduceRegion(
    reducer=ee.Reducer.mean(),
    geometry=point, scale=10
).get("B8").getInfo()

ndbi_val = ndbi.reduceRegion(
    reducer=ee.Reducer.mean(),
    geometry=point, scale=10
).get("B11").getInfo()

return ndvi_val or 0, ndbi_val or 0

# ----- #
Sidebar
# -----
st.sidebar.header("Enter Location")

lat = st.sidebar.number_input("Latitude",
    value=float(data["Latitude_x"].mean())) lon
= st.sidebar.number_input("Longitude",
    value=float(data["Longitude_x"].mean()))

date = st.sidebar.date_input("Select Date")

time_slot = st.sidebar.selectbox(

```



```

    "Select Time",
    ["Morning (9-12)", "Afternoon (12-3)", "Evening (3-6)"]
)

predict_button = st.sidebar.button("Predict")

# -----
# Heatmap
# -----
st.subheader("Urban Heat Map")

m = folium.Map(
    location=[data["Latitude_x"].mean(),
data["Longitude_x"].mean()],
    zoom_start=11,
    tiles="CartoDB
Voyager"
)

HeatMap(
    data[["Latitude_x", "Longitude_x", "Temp_C"]].values.tolist(),
    radius=35,

    blur=25
).add_to(m)

st_folium(m, width=900)

# ----- #
Prediction
# -----
if predict_button:

    ndvi, ndbi = get_indices(lat, lon, date)

    prediction = model.predict(pd.DataFrame(
        [[lat, lon, ndvi, ndbi, date.year, encode_time(time_slot)]],
        columns=["Latitude_x", "Longitude_x", "NDVI", "NDBI", "Year", "Time"]
    ))[0]

    vulnerability = calculate_vulnerability(prediction, ndvi, ndbi)

```

```

    st.session_state.prediction = (prediction, vulnerability)
st.session_state.time = time.time()

# -----
# Output (ONLY 1 MINUTE)
# -----
if st.session_state.prediction is not None:

    if time.time() - st.session_state.time < 60:

        prediction, vulnerability = st.session_state.prediction
        risk = classify_risk(vulnerability)

        st.success(f" 🌡 Temperature: {prediction:.2f} °C")
        st.metric("Vulnerability Score", vulnerability)
        st.write(f" ⚠ Risk Level: {risk}")

        if risk == "High Risk":
            st.error(" 🚨 Severe Heat Alert")
        elif risk == "Moderate Risk":
            st.warning(" ⚠ Moderate Risk")
        else:
            st.success(" ✅ Safe Zone")

    # Map
    result_map = folium.Map(location=[lat, lon],
zoom_start=14)

    color = "red" if risk=="High Risk" else "orange" if risk=="Moderate Risk"
    else "green"

    folium.Marker(
        [lat, lon],
        popup=f" {prediction:.2f}
°C",
        icon=folium.Icon(color=color)
    ).add_to(result_map)

```

```
st_folium(result_map, width=900)
```

```
else:
```

```
st.warning("⌚ Result expired! Click Predict again.")
```

OUTPUT:

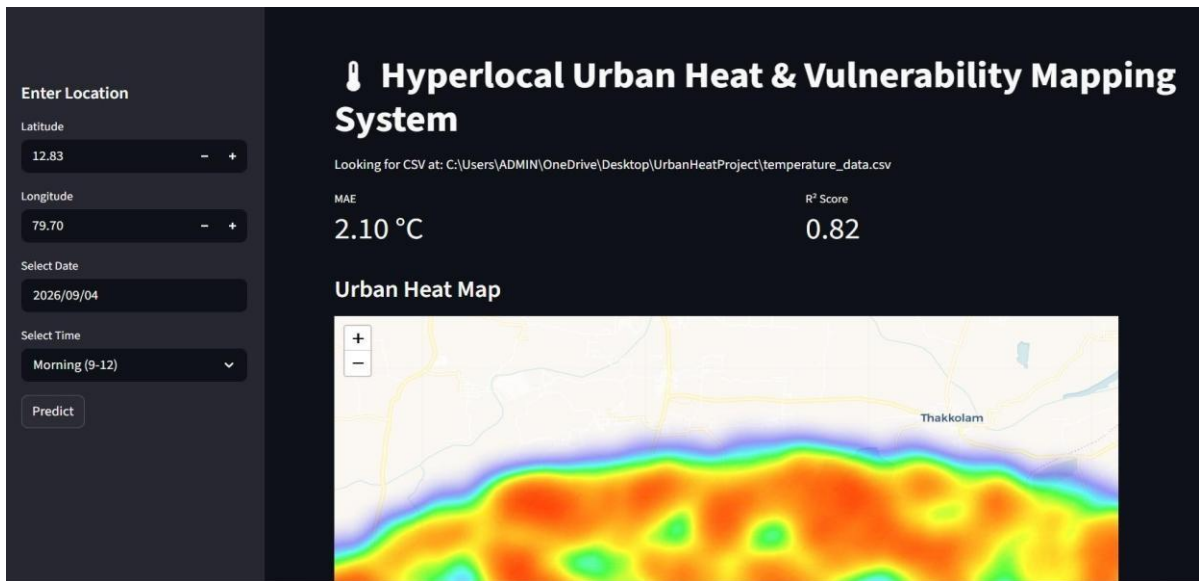


Fig: 8.1 Getting Input from User

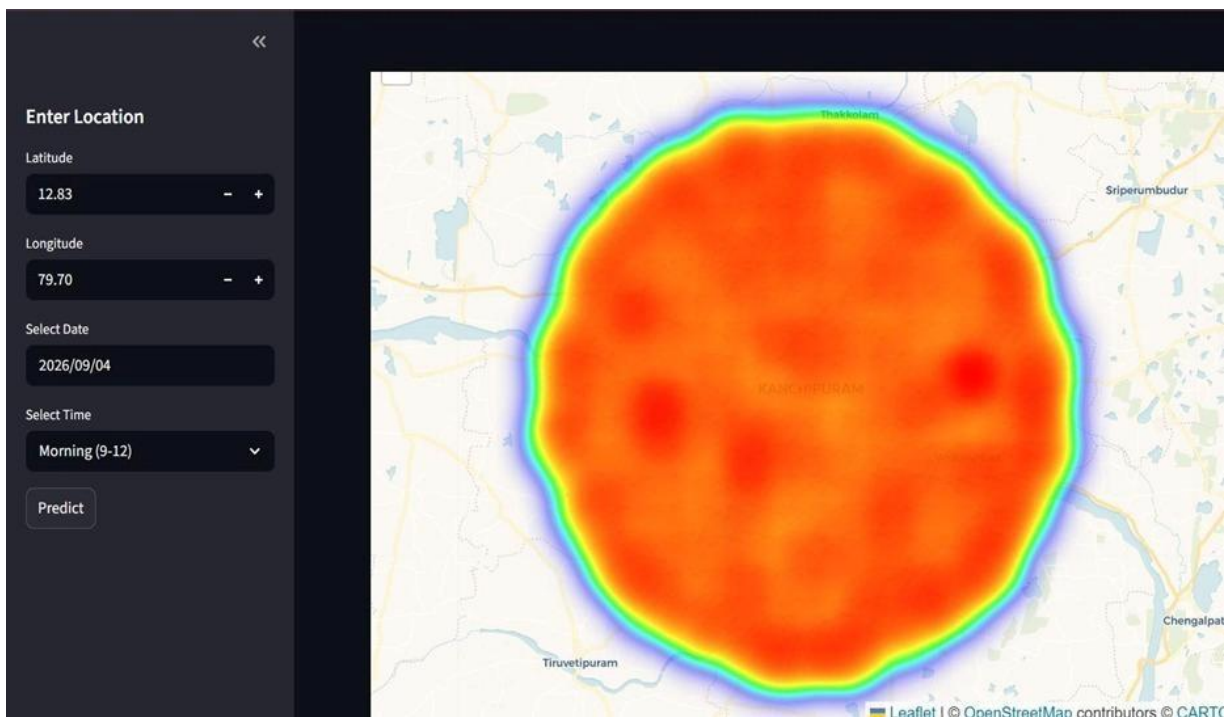


Fig: 8.2 Heatmap for Study Area

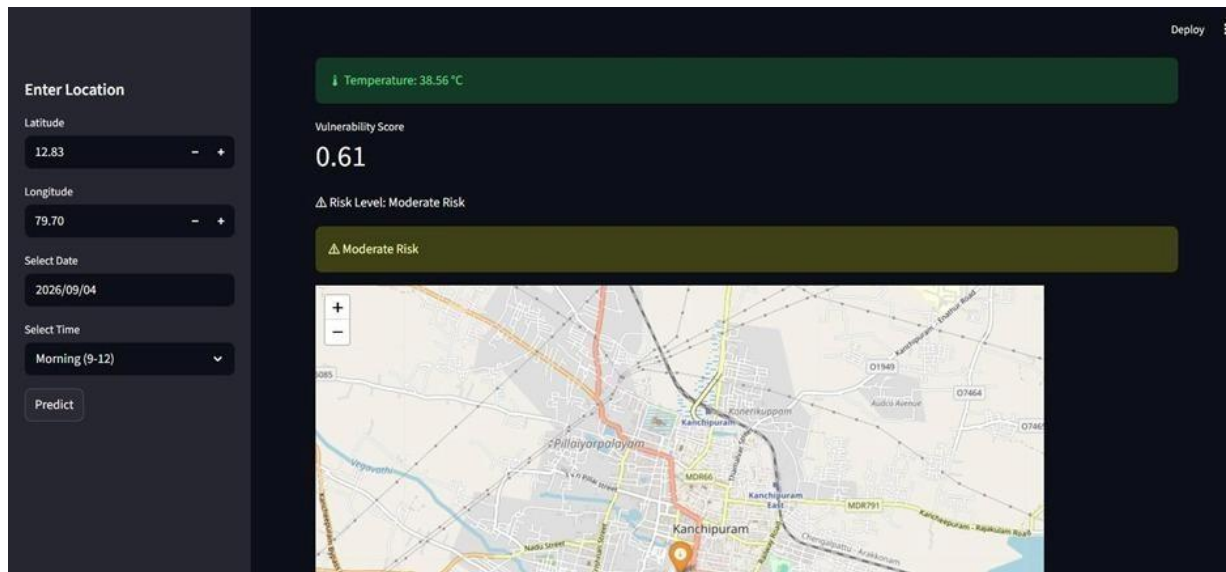


Fig: 8.3 Vulnerability Score & Alert Message

CHAPTER 9

SYSTEM TESTING

System testing is an essential phase in the software development life cycle that ensures the system functions correctly and meets the required specifications. It involves verifying and validating different components of the system to identify and fix errors.

9.1 Unit Testing

Unit testing involves testing individual modules or components of the system independently to ensure that each unit performs its intended function correctly.

In this project, the following modules are tested individually:

- **Data Collection Module:**
Verified whether satellite data is correctly retrieved and stored.
- **Data Processing Module:**
Checked for proper handling of missing values, normalization, and feature extraction.
- **Machine Learning Module:**
Ensured that the model is trained correctly and produces accurate predictions.
- **Visualization Module:**
Tested the generation of maps and heatmaps for correctness.
- **Alert Module:**
Verified that alerts are triggered when temperature exceeds the threshold.

Advantages of Unit Testing

- Helps identify errors at an early stage
- Improves code reliability
- Simplifies debugging process

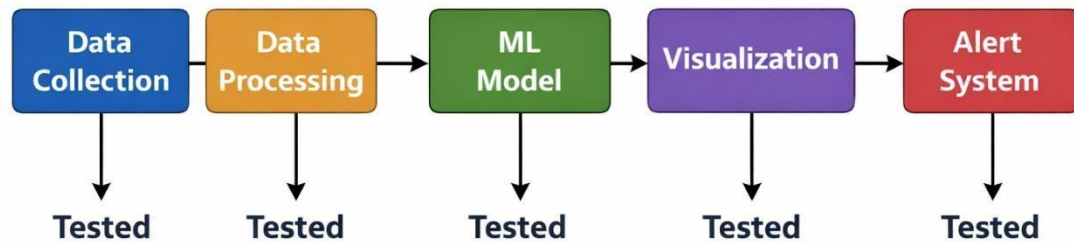


Fig 9.1 Unit Testing

9.2 Integration Testing

Integration testing involves combining all modules and testing the system as a whole to ensure smooth interaction between components.

In this project, integration testing ensures:

- Proper data flow between modules
- Correct integration of machine learning model with visualization
- Seamless communication between backend and frontend
- Accurate display of results in the user interface
- Input data → processed → predicted → visualized correctly
- Heatmap displays correct temperature distribution
- Alerts are triggered and displayed properly

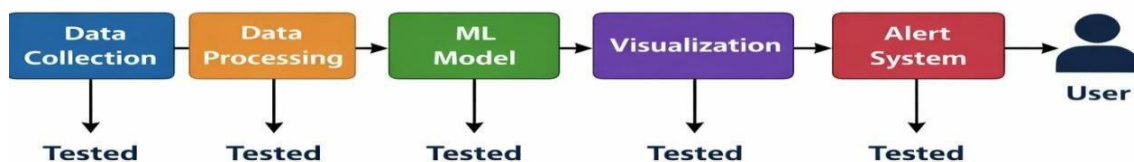


Fig 9.2 Integration Testing

CHAPTER 10

SYSTEM IMPLEMENTATION

System implementation refers to the process of transforming the designed system into a fully functional application. In this project, the implementation is carried out using modern tools and technologies to ensure high efficiency, scalability, and user-friendliness.

Implementation Details

- The application is developed using **Streamlit**, which provides an interactive and user-friendly web interface for users to interact with the system.
- The machine learning model, specifically the **Random Forest algorithm**, is implemented using **Python** and the **Scikit-learn** library for accurate prediction.
- Satellite data is collected and processed using **Google Earth Engine**, which converts raw satellite imagery into meaningful input features for the model.
- Heatmaps and spatial visualizations are generated using **Folium** along with **OpenStreetMap**, enabling clear geographic representation of temperature variations.

Deployment

- The system is deployed locally on a computer system.
- Users can access the application through a standard web browser.
- The interface allows users to input parameters and view prediction results in real-time, ensuring ease of use and quick analysis.

Key Features of Implementation

- Interactive dashboard for user engagement
- Real-time visualization of heat patterns
- Accurate prediction using machine learning techniques

CHAPTER 11

CONCLUSION AND FUTURE ENHANCEMENT

11.1 Conclusion

The project “Hyperlocal Urban Heat Vulnerability Mapping and Alert System” successfully demonstrates the application of machine learning and geospatial technologies in analyzing urban heat patterns.

The system effectively:

- Collects and processes satellite data
- Predicts temperature using Random Forest algorithm
- Visualizes results through interactive heatmaps
- Identifies heat-vulnerable zones
- Provides alerts for high-temperature conditions

The integration of multiple technologies such as Python, Streamlit, and geospatial tools has resulted in a robust and efficient system. The model evaluation using MAE indicates that the predictions are reasonably accurate.

Overall, the system provides a practical solution for understanding urban heat distribution and supports better decision-making for urban planning and environmental management.

11.2 Future Enhancement

Although the system performs effectively, several enhancements can be made to improve its functionality and scalability:

- **Real-Time Data Integration:**
Incorporating real-time weather data for more accurate predictions.
- **Mobile Application Development:**
Developing a mobile app to increase accessibility and usability.
- **IoT Integration:**
Using IoT-based temperature sensors for real-time monitoring.

- **Advanced Machine Learning Models:**
Implementing deep learning models for improved accuracy.
- **Multi-City Expansion:**
Extending the system to cover multiple cities and regions.
- **Automated Alert System:**
Sending alerts via SMS, email, or notifications.

These enhancements can significantly improve the system's performance and applicability in real-world scenarios.

CHAPTER 12

BIBLIOGRAPHY

The development of this project is supported by various research papers, technical documentation, and online resources related to urban heat analysis and geospatial technologies. Studies on urban heat islands and satellite-based temperature mapping have provided insights into spatial heat variations. Data and methodologies from NASA and European Space Agency have been useful for satellite data analysis.

Machine learning concepts referenced from IEEE publications have contributed to predictive modeling. Reports from the World Health Organization helped in understanding heat-related health risks and early warning systems. Tools such as Google Earth Engine and QGIS supported data processing and visualization.

These resources collectively ensured that the project is scientifically accurate, technically feasible, and aligned with modern research in climate and smart city applications.

CHAPTER 13

REFERENCES

- [1] F. Larroya et al., “Hyperlocal Urban Heat Mapping using Satellite Data,” *Remote Sensing of Environment*, vol. 310, 2025.
- [2] Institute of Electrical and Electronics Engineers, “Machine Learning in Heat Vulnerability Mapping,” *IEEE Access*, vol. 11, pp. 56789–56805, 2023.
- [3] World Health Organization, “Heat Risk Early Warning Systems,” *WHO Climate and Health Technical Report Series*, 2023.
- [4] P. Siddiqui et al., “Urban Heat Vulnerability Mapping using GIS and Remote Sensing,” *Sustainable Cities and Society*, vol. 85, 2022.
- [5] S. C. Sheridan and L. S. Kalkstein, “Heat Vulnerability Index: Advancements and Applications,” *International Journal of Environmental Research and Public Health*, vol. 19, no. 14, 2022.
- [6] NASA, “Landsat 8 Data Users Handbook,” 2022.
- [7] European Space Agency, “Sentinel-2 User Guide,” 2023.
- [8] J. A. Voogt and T. R. Oke, “Thermal remote sensing of urban climates,” *Remote Sensing of Environment*, vol. 86, no. 3, pp. 370–384, 2003.
- [9] H. Akbari, M. Pomerantz, and H. Taha, “Cool surfaces and shade trees to reduce energy use and improve air quality in urban areas,” *Solar Energy*, vol. 70, no. 3, pp. 295–310, 2001.
- [10] S. K. Ghosh and P. Das, “Urban Heat Island Analysis Using Remote Sensing and GIS Techniques,” *International Journal of Environmental Science and Technology*, vol. 17, pp. 123–135, 2020.
- [11] T. R. Oke, “The energetic basis of the urban heat island,” *Quarterly Journal of the Royal Meteorological Society*, vol. 108, no. 455, pp. 1–24, 1982.
- [12] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, Cambridge, MA: MIT Press, 2016.

Web References:

- [1] NASA, “Landsat Missions,” Available: <https://landsat.gsfc.nasa.gov/>
- [2] European Space Agency, “Sentinel-2 Overview,” Available: <https://sentinels.copernicus.eu/>
- [3] Google Earth Engine, “Earth Engine Documentation,” Available: <https://developers.google.com/earth-engine>
- [4] QGIS, “QGIS Documentation,” Available: <https://docs.qgis.org/>
- [5] World Health Organization, “Heatwaves and Health Guidance,” Available: <https://www.who.int/>
- [6] United States Geological Survey, “Land Surface Temperature Data,” Available: <https://www.usgs.gov/>
- [7] Copernicus Programme, “Copernicus Climate Data Store,” Available: <https://climate.copernicus.eu/>
- [8] National Oceanic and Atmospheric Administration, “Climate Data Online,” Available: <https://www.noaa.gov/>